

Guia Simples dos Algoritmos

O que cada algoritmo faz, como funciona e como eles se comparam · Análise de Algoritmos · Prova de 22/09/2026

1. Mapa geral: todos os algoritmos em uma página

2. Algoritmos de busca

- 2.1 Busca linear (sequencial)
- 2.2 Busca binária
- 2.3 Comparativo: busca linear × busca binária

3. Algoritmos de ordenação

- 3.1 Selection sort (ordenação por seleção)
- 3.2 Bubble sort (ordenação por bolha)
- 3.3 Quicksort
- 3.4 Merge sort (ordenação por intercalação)
- 3.5 Comparativo dos quatro algoritmos de ordenação

4. Recursão e as três versões do Fibonacci

- 4.1 O que é recursão, em termos simples
- 4.2 A sequência de Fibonacci
- 4.3 Comparativo das três versões

5. Dividir para conquistar

- 5.1 A estratégia
- 5.2 Euclides: o maior quadrado da fazenda (MDC)

6. Força bruta: o caixeiro-viajante

7. Comparativo geral

- 7.1 Quanto tempo cada classe leva, na prática
- 7.2 Qual algoritmo usar em cada situação
- 7.3 Os mesmos algoritmos, agrupados por ideia
- 7.4 Cinco frases para levar

Para que serve este guia. Explicar, em linguagem simples, **o que cada algoritmo visto no curso faz**, com uma analogia do dia a dia, o passo a passo, um exemplo pequeno com a saída real e o custo (Big O). No fim de cada grupo há um **comparativo** entre os algoritmos do grupo, e a última parte compara **todos** entre si. Os exercícios ficam na *Revisão* e o código completo fica no *Catálogo de Scripts*; aqui é só a teoria.

1. Mapa geral: todos os algoritmos em uma página

Algoritmo	Tipo	O que faz, em uma frase	Melhor	Médio	Pior	Memória extra
Busca linear	busca	olha um item por vez até achar	$O(1)$	$O(n)$	$O(n)$	$O(1)$
Busca binária	busca	abre no meio de uma lista ordenada e descarta metade a cada passo	$O(1)$	$O(\log n)$	$O(\log n)$	$O(1)$
Selection sort	ordenação	acha o menor e o coloca na frente, repete	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$
Bubble sort	ordenação	compara vizinhos e troca; o maior “sobe” para o fim	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Quicksort	ordenação (dividir para conquistar)	escolhe um pivô, separa menores e maiores, repete em cada lado	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	pilha $O(\log n)$ a $O(n)$
Merge sort	ordenação (dividir para conquistar)	divide ao meio até sobrar 1, depois junta as metades já ordenadas	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$
Fibonacci recursivo puro	recursão	$F(n) = F(n-1) + F(n-2)$ recalculando tudo	—	$O(2^n)$	—	pilha $O(n)$
Fibonacci com memorização	recursão + cache	igual, mas guarda cada resultado num dicionário	—	$O(n)$	—	$O(n)$
Fibonacci iterativo	laço	soma de baixo para cima com duas variáveis	—	$O(n)$	—	$O(1)$
Euclides (maior quadrado / MDC)	dividir para conquistar	troca o problema pela “sobra” até um lado dividir o outro	—	$O(\log n)$	—	pilha $O(\log n)$
Caixeiro-viajante (força bruta)	força bruta	testa todas as rotas possíveis	—	$O(n!)$	—	$O(n)$

Ordem das classes, da mais rápida para a mais lenta: $O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(2^n) < O(n!)$

2. Algoritmos de busca

Problema: **achar um valor dentro de uma lista** (e dizer em que posição ele está, ou que não está).

2.1 Busca linear (sequencial)

Em uma frase: olha a lista do começo ao fim, um item por vez, até encontrar o que procura.

Analogia: procurar uma palavra em um caderno sem índice, lendo página por página.

Como funciona:

1. Começa no primeiro item.
2. Compara com o valor procurado. Se for igual, achou: para.
3. Se não, passa para o próximo item e repete.
4. Se chegou ao fim sem achar, o valor não está na lista.

Exemplo (procurando 37 em [3, 8, 12, 19, 25, 31, 37, 42, 49, 56]):

```
passo 1: 3    nao e 37
passo 2: 8    nao e 37
passo 3: 12   nao e 37
passo 4: 19   nao e 37
passo 5: 25   nao e 37
passo 6: 31   nao e 37
passo 7: 37   ACHOU no indice 6 -> 7 passos
```

Custo: melhor caso 1 passo (o item é o primeiro); pior caso n passos (é o último ou não existe). Em média, $n/2$. Classe **$O(n)$** : se a lista dobra, o trabalho dobra.

Pontos fortes: funciona em **qualquer lista**, ordenada ou não; não precisa de preparação; é trivial de escrever. **Pontos fracos:** lenta para listas grandes (1 milhão de itens $\rightarrow$ até 1 milhão de passos). **Quando usar:** listas pequenas, listas desordenadas, ou quando você vai buscar **uma única vez** (ordenar antes custaria mais).

2.2 Busca binária

Em uma frase: em uma lista **já ordenada**, abre no meio, vê se o valor procurado é menor ou maior, e joga fora a metade que não serve; repete até achar.

Analogia: procurar uma palavra no dicionário. Você abre no meio, vê que a palavra está antes, então ignora toda a segunda metade, e assim por diante. Ou o jogo de adivinhar um número de 1 a 100: chutando sempre o meio, você acerta em no máximo 7 tentativas.

Como funciona:

1. Marca o início (baixo) e o fim (alto) da parte da lista que ainda interessa.
2. Olha o item do **meio**.
3. Se for o valor procurado, achou.
4. Se o meio for **menor** que o valor, o valor só pode estar à **direita**: move baixo para meio + 1.
5. Se o meio for **maior**, o valor só pode estar à **esquerda**: move alto para meio - 1.
6. Repete até achar ou até baixo passar de alto (o valor não está na lista).

Exemplo (mesma lista, procurando 37):

```
passo 1: baixo=0 alto=9 meio=4 -> 25 < 37, vai para a direita
passo 2: baixo=5 alto=9 meio=7 -> 42 > 37, vai para a esquerda
passo 3: baixo=5 alto=6 meio=5 -> 31 < 37, vai para a direita
passo 4: baixo=6 alto=6 meio=6 -> 37 ACHOU no indice 6 -> 4 passos
```

Custo: a cada passo a lista cai pela metade, então o número de passos é “quantas vezes dá para dividir n por 2”: $\log_2 n$. Para 128 itens, 7 passos; para 1.000.000, 20 passos; para 4 bilhões, 32. Classe **$O(\log n)$** : se a lista **dobra**, custa **um passo a mais**.

Pontos fortes: absurdamente rápida em listas grandes. **Pontos fracos:** **exige lista ordenada**. Se a lista não está ordenada, a comparação com o meio não diz nada e o algoritmo pode descartar a metade certa. Ordenar antes custa $O(n \log n)$. **Quando usar:** lista grande, ordenada (ou que você mantém ordenada), com **muitas buscas**.

2.3 Comparativo: busca linear × busca binária

	Busca linear	Busca binária
Exige lista ordenada?	não	sim
Passos, pior caso, $n = 10$	10	4
Passos, pior caso, $n = 1.000$	1.000	10
Passos, pior caso, $n = 1.000.000$	1.000.000	20
Se n dobra...	dobro o trabalho	+1 passo
Melhor caso	item na 1ª posição (1 passo)	item bem no meio (1 passo)
Tempo medido na atividade 01 ($n = 1$ milhão, pior caso)	≈ 30 ms	≈ 0,01 ms

Resumo comparativo. A binária ganha de longe quando a lista é grande **e** está ordenada. A linear ganha quando a lista é pequena, quando está desordenada e você vai buscar só uma vez, ou no caso particular em que o item está logo no começo. Por isso a pergunta certa não é “qual é melhor?”, e sim “**quantas vezes vou buscar?**”: se for muitas, vale pagar a ordenação uma vez e usar a binária sempre.

3. Algoritmos de ordenação

Problema: **colocar uma lista em ordem** (crescente, no nosso caso). Os quatro algoritmos vistos chegam ao mesmo resultado; o que muda é **quanto trabalho** fazem e **como** se comportam conforme a lista de entrada.

3.1 Selection sort (ordenação por seleção)

Em uma frase: procura o **menor** elemento da parte desordenada e o coloca na primeira posição livre; repete até acabar.

Analogia: organizar cartas na mão: você olha todas, pega a menor e põe na frente; olha as que sobraram, pega a menor e põe em seguida; e assim por diante.

Como funciona:

1. Percorre toda a lista e encontra o menor valor.
2. Troca esse menor com o item da **posição 0**.
3. Percorre da posição 1 em diante, encontra o menor e troca com a posição 1.
4. Repete até a penúltima posição. A última já fica certa sozinha.

Exemplo ([5, 2, 9, 1, 7]):

```
passagem 1: menor=1 -> troca com o 5 -> [1, 2, 9, 5, 7] 4 comparacoes
passagem 2: menor=2 -> ja esta no lugar -> [1, 2, 9, 5, 7] 3 comparacoes
passagem 3: menor=5 -> troca com o 9 -> [1, 2, 5, 9, 7] 2 comparacoes
passagem 4: menor=7 -> troca com o 9 -> [1, 2, 5, 7, 9] 1 comparacao
total: 10 comparacoes, 3 trocas
```

Custo: para achar o menor de n itens faz $n-1$ comparações, depois $n-2$, depois $n-3$... Total $n(n-1)/2$, que é $O(n^2)$. E isso acontece **sempre**, mesmo se a lista já estiver ordenada, porque ele não “percebe” a ordem: varre tudo do mesmo jeito.

Pontos fortes: simples; faz **poucas trocas** (no máximo $n-1$), então é bom quando mover um item é caro (registros grandes). **Pontos fracos:** $O(n^2)$ em todos os casos; não aproveita lista quase ordenada.

3.2 Bubble sort (ordenação por bolha)

Em uma frase: compara cada par de **vizinhos** e troca se estiverem fora de ordem; a cada passagem o maior elemento “borbulha” até o fim.

Analogia: bolhas subindo na água: a maior bolha sobe até a superfície na primeira passagem; a segunda maior, na próxima. Se uma passagem inteira não troca nada, a água está “calma”: a lista já está ordenada e o algoritmo para.

Como funciona:

1. Compara o item 0 com o 1; se o 0 for maior, troca. Depois compara o 1 com o 2, e assim até o fim. Ao terminar, o maior está na última posição.
2. Repete a passagem, agora ignorando a última posição (já está certa).
3. Se em alguma passagem **não houve troca**, para: a lista está ordenada.

Exemplo ([5, 2, 9, 1, 7]):

```
passagem 1: (5,2) troca (5,9) ok (9,1) troca (9,7) troca -> [2, 5, 1, 7, 9]
passagem 2: (2,5) ok (5,1) troca (5,7) ok -> [2, 1, 5, 7, 9]
passagem 3: (2,1) troca (2,5) ok -> [1, 2, 5, 7, 9]
passagem 4: (1,2) ok -> nenhuma troca: PARA
total: 10 comparacoes, 5 trocas, 4 passagens
```

Custo: pior caso (lista invertida) $n(n-1)/2$ comparações e $n(n-1)/2$ trocas: $O(n^2)$. Melhor caso (lista já ordenada): uma passagem com $n-1$ comparações e nenhuma troca: $O(n)$. É o único dos quatro que tem melhor caso linear.

Pontos fortes: muito simples; detecta lista já ordenada e para cedo; ótimo para listas **quase ordenadas**. **Pontos fracos:** faz muitas trocas; $O(n^2)$ em lista aleatória ou invertida.

3.3 Quicksort

Em uma frase: escolhe um item como **pivô**, separa a lista em “menores que o pivô” e “maiores que o pivô”, e ordena cada parte do mesmo jeito (recursivamente); no fim junta menores + pivô + maiores.

Analogia: separar uma pilha de provas por nota escolhendo uma nota de referência (o pivô): quem tirou menos vai para a esquerda, quem tirou mais vai para a direita. Depois faz a mesma coisa dentro de cada pilha, até sobraem pilhas de uma prova só, que já estão “ordenadas”.

Como funciona:

1. **Caso base:** lista com 0 ou 1 item já está ordenada; devolve como está.
2. Escolhe o pivô (na aula: o primeiro item).
3. **Particiona:** monta uma lista com os menores que o pivô e outra com os maiores.
4. Chama o quicksort em cada uma das duas listas.
5. **Combina:** menores ordenados + [pivô] + maiores ordenados.

Exemplo ([33, 15, 10, 42, 7], pivô = primeiro):

```
quicksort([33, 15, 10, 42, 7])  pivô 33 -> menores [15, 10, 7]  maiores [42]
quicksort([15, 10, 7])          pivô 15 -> menores [10, 7]    maiores []
quicksort([10, 7])              pivô 10 -> menores [7]         maiores []
-> [7] + [10] + [] = [7, 10]
-> [7, 10] + [15] + [] = [7, 10, 15]
-> [7, 10, 15] + [33] + [42] = [7, 10, 15, 33, 42]
7 chamadas, pilha com 4 níveis
```

Custo: cada “nível” da recursão toca todos os n itens (para particionar), então o custo total é $n \times \text{número de níveis}$. Se o pivô divide a lista mais ou menos ao meio, são $\log_2 n$ níveis: **$O(n \log n)$** . Se o pivô cai sempre numa ponta (lista já ordenada com pivô = primeiro item), um dos lados fica vazio, são n níveis: **$O(n^2)$** , o pior caso. Escolhendo o pivô do meio ou um aleatório, o pior caso praticamente não acontece.

Pontos fortes: muito rápido na prática (é o qsort da linguagem C); constante pequena; usa pouca memória extra em relação ao merge sort. **Pontos fracos:** pior caso $O(n^2)$ se o pivô for mal escolhido; recursão profunda pode estourar a pilha (RecursionError).

3.4 Merge sort (ordenação por intercalação)

Em uma frase: divide a lista ao meio, e cada metade ao meio, até sobraem listas de 1 item; depois vai **juntando** (intercalando) pares de listas já ordenadas em listas maiores ordenadas.

Analogia: dois baralhos já ordenados podem ser juntados em um só olhando apenas a carta de cima de cada um e pegando sempre a menor. O merge sort divide o baralho até ter 52 “baralhos” de uma carta (cada um trivialmente ordenado) e vai juntando de dois em dois.

Como funciona:

1. **Caso base:** lista com 0 ou 1 item já está ordenada.
2. **Divide** a lista em duas metades.
3. Chama o merge sort em cada metade.
4. **Intercala (merge):** com um dedo no início de cada metade, copia sempre o menor dos dois para a lista final, avança o dedo e repete até acabar. Isso custa $O(n)$ por junção.

Exemplo ([38, 27, 43, 3, 9, 82, 10]):

```

divide: [38, 27, 43] e [3, 9, 82, 10]
divide: [38] [27, 43] [3, 9] [82, 10]
divide: [27] [43] [3] [9] [82] [10]
junta: [27] + [43] -> [27, 43]
junta: [38] + [27, 43] -> [27, 38, 43]
junta: [3] + [9] -> [3, 9]
junta: [82] + [10] -> [10, 82]
junta: [3, 9] + [10, 82] -> [3, 9, 10, 82]
junta: [27, 38, 43] + [3, 9, 10, 82] -> [3, 9, 10, 27, 38, 43, 82]
13 comparações, pilha com 4 níveis

```

Custo: a divisão ao meio dá **sempre** $\log_2 n$ níveis, e cada nível intercala n itens: **$O(n \log n)$ em todos os casos**, não importa a ordem inicial. Em troca, precisa de **memória extra $O(n)$** para as listas intercaladas.

Pontos fortes: desempenho **garantido** $O(n \log n)$ no pior caso; é estável (itens iguais mantêm a ordem original). **Pontos fracos:** usa mais memória; a constante é maior que a do quicksort, então na prática costuma ser um pouco mais lento em listas aleatórias.

3.5 Comparativo dos quatro algoritmos de ordenação

Operações medidas com $n = 5$ (listas da atividade 02):

Lista de entrada	Selection: comparações / trocas	Bubble: comparações / trocas
[1, 2, 3, 4, 5] já ordenada	10 / 0	4 / 0 (para na 1ª passagem)
[3, 5, 1, 4, 2] aleatória	10 / 2	10 / 6
[5, 4, 3, 2, 1] invertida	10 / 2	10 / 10

Comparações medidas conforme n cresce (mesma lista aleatória para todos, semente fixa; quicksort com pivô aleatório, média de 5 execuções):

n	Selection	Bubble (aleatória)	Bubble (invertida)	Quicksort	Merge sort	$n \cdot \log_2 n$	$n(n-1)/2$
25	300	245	300	98	89	116	300
100	4.950	4.929	4.950	627	555	664	4.950
400	79.800	79.394	79.800	3.631	2.943	3.458	79.800
1.000	499.500	498.834	499.500	11.335	8.702	9.966	499.500

Leitura: os dois $O(n^2)$ crescem **4×** quando n dobra ($300 \rightarrow 1.225 \rightarrow 4.950 \rightarrow 19.900$); os dois $O(n \log n)$ crescem pouco mais que 2×. Em $n = 1.000$ a diferença já é de **cerca de 50 vezes**; em $n = 1.000.000$ seria de 25.000 vezes.

Caso especial que muda tudo: a lista já ordenada

Algoritmo	Lista já ordenada, $n = 1.000$	Por quê
Bubble sort	999 comparações ($O(n)$)	percebe que não trocou nada e para
Selection sort	499.500 comparações	não olha se está ordenada
Quicksort, pivô = primeiro	499.500 comparações, pilha com 1.000 níveis	um lado sempre vazio: pior caso
Quicksort, pivô do meio	7.987 comparações, pilha com 10 níveis	divide ao meio
Merge sort	4.932 comparações, pilha com 11 níveis	sempre divide ao meio; em lista ordenada cada intercalação para na metade

Quadro-resumo

	Selection	Bubble	Quicksort	Merge sort
Ideia	pega o menor	troca vizinhos	pivô e partição	divide e intercala
Melhor / médio / pior	$n^2 / n^2 / n^2$	$n / n^2 / n^2$	$n \log n / n \log n / n^2$	$n \log n / n \log n / n \log n$
Memória extra	$O(1)$	$O(1)$	pilha $\log n$ a n	$O(n)$
Trocas / movimentações	poucas ($\leq n-1$)	muitas	médias	copia tudo a cada nível
Sensível à ordem inicial?	não	sim (ordenada = ótimo)	sim (ordenada + pivô fixo = pior)	não
Usa dividir para conquistar?	não	não	sim	sim
Quando usar	lista pequena e troca cara	lista pequena ou quase ordenada	listas grandes em geral	quando precisa garantir o pior caso ou estabilidade

Resumo comparativo. Selection e bubble são os “simples e lentos”: servem para listas pequenas. Entre os dois, o bubble vence se a lista está quase ordenada (ele para cedo) e o selection vence se trocar itens é caro (ele quase não troca). Quicksort e merge sort são os “rápidos”: ambos $O(n \log n)$ no caso médio porque **dividem o problema**. O quicksort costuma ser o mais rápido na prática, mas tem um pior caso $O(n^2)$ que se evita escolhendo bem o pivô. O merge sort nunca tem pior caso ruim, mas paga com memória extra.

4. Recursão e as três versões do Fibonacci

4.1 O que é recursão, em termos simples

Uma função **recursiva** é uma função que **chama a si mesma** com um problema menor, até chegar a um caso tão simples que a resposta é imediata.

- **Caso base:** o caso simples, resolvido sem nova chamada (lista vazia $\rightarrow$ soma 0; $n \leq 1 \rightarrow$ fatorial 1).
- **Caso recursivo:** a função chama a si mesma com um pedaço menor do problema e usa a resposta para montar a sua.

Cada chamada em andamento ocupa um “quadro” na **pilha de chamadas**. A profundidade da recursão é a altura dessa pilha. Sem caso base, a pilha cresce para sempre e o programa quebra (RecursionError em Python, com limite padrão de cerca de 1.000 chamadas).

Exemplo: soma de uma lista.

```
soma([2, 4, 6]) = 2 + soma([4, 6])
                = 2 + (4 + soma([6]))
                = 2 + (4 + (6 + soma([])))    <- caso base: 0
                = 2 + (4 + (6 + 0)) = 12
```

Recursão não é mais rápida que um laço; ela é usada porque deixa certos problemas **muito mais fáceis de escrever**, em especial os de dividir para conquistar.

4.2 A sequência de Fibonacci

Cada número é a soma dos dois anteriores: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144... Em fórmula: $F(0) = 0$, $F(1) = 1$, $F(n) = F(n-1) + F(n-2)$. É o problema clássico para mostrar três jeitos de programar a mesma coisa com custos completamente diferentes.

Versão A: recursiva pura

Em uma frase: traduz a fórmula direto para código: $\text{fib}(n) = \text{fib}(n-1) + \text{fib}(n-2)$.

O problema: ela **recalcula** os mesmos valores muitas vezes. Para calcular $\text{fib}(5)$, $\text{fib}(3)$ é calculado 2 vezes, $\text{fib}(2)$ 3 vezes e $\text{fib}(1)$ 5 vezes:

```
fib(5)
|- fib(4)
|  |- fib(3)
|   |  |- fib(2) -> fib(1), fib(0)
|   |  |- fib(1)
|   |- fib(2) -> fib(1), fib(0)
|- fib(3)
|  |- fib(2) -> fib(1), fib(0)
|- fib(1)
15 chamadas para um problema de tamanho 5
```

Custo: o número de chamadas cresce como $1,618^n$ (a razão áurea): **$O(2^n)$, exponencial**. Cada +1 em n multiplica o trabalho por 1,6. Para $n = 30$ são 2,7 milhões de chamadas; para $n = 40$, mais de 300 milhões. Memória: só a pilha, $O(n)$, porque as chamadas não ficam todas vivas ao mesmo tempo.

Versão B: com memorização (cache)

Em uma frase: igual à recursiva, mas **guarda cada resultado num dicionário**; antes de calcular, consulta o dicionário e, se já tem, devolve na hora.

Analogia: fazer uma conta difícil uma vez e anotar o resultado num post-it; nas próximas vezes, em vez de refazer a conta, lê o post-it.

Como funciona:

1. Se n já está no dicionário, devolve o valor guardado (custo $O(1)$).
2. Senão, calcula $\text{fib}(n-1) + \text{fib}(n-2)$, guarda no dicionário e devolve.

Custo: cada valor é calculado **uma única vez**, então o total é linear: exatamente $2n - 1$ chamadas. **$O(n)$** de tempo, **$O(n)$** de memória para o dicionário (mais a pilha). Em Python, `@lru_cache` faz isso automaticamente.

Versão C: iterativa (de baixo para cima)

Em uma frase: começa de $F(0)$ e $F(1)$ e vai somando para a frente com duas variáveis, sem recursão nem dicionário.

```
anterior=0 atual=1
passo 1: anterior=1 atual=1
passo 2: anterior=1 atual=2
passo 3: anterior=2 atual=3
passo 4: anterior=3 atual=5
passo 5: anterior=5 atual=8    -> fib(5) = 5 (o 'anterior' apos 5 passos)
```

Custo: um laço de n voltas: **$O(n)$** de tempo e **$O(1)$** de memória. Não usa pilha, então não corre risco de RecursionError.

4.3 Comparativo das três versões

Chamadas de função

n	Recursiva pura	Com memorização ($2n - 1$)	Iterativa (voltas do laço)
5	15	9	5
10	177	19	10
20	21.891	39	20
30	2.692.537	59	30

Tempo medido (atividade 03 e catálogo de scripts):

n	Recursiva pura	Com memorização	Iterativa
20	≈ 0,002 s	≈ 0,000004 s	≈ 0,0000004 s
28	≈ 0,09 s	≈ 0,000006 s	≈ 0,0000005 s
32	≈ 0,5 s	≈ 0,000006 s	≈ 0,000001 s
40	≈ 16 s (estimado)	≈ 0,00001 s	≈ 0,000001 s

Quadro-resumo

	Recursiva pura	Com memorização	Iterativa
Tempo	$O(2^n)$	$O(n)$	$O(n)$
Memória	pilha $O(n)$	dicionário $O(n)$ + pilha $O(n)$	$O(1)$
Risco de estourar a pilha	sim, para n grande	sim ($n \approx 1.000+$)	não
Facilidade de escrever	a mais simples	simples (<code>@lru_cache</code>)	simples
Quando usar	nunca em produção; só para n pequeno ou para ensinar	quando há subproblemas repetidos e chamadas repetidas ao longo do programa	quando se quer o mais eficiente em tempo e memória

Resumo comparativo. As três dão o mesmo resultado; a diferença está em **quantas vezes o mesmo subproblema é resolvido**. A recursiva pura resolve o mesmo $\text{fib}(k)$ milhares de vezes; a memorização resolve cada um uma vez e reaproveita; a iterativa resolve cada um uma vez e nem precisa guardar, porque avança em ordem. A lição geral: quando um algoritmo recursivo tem **subproblemas sobrepostos**, guardar resultados (cache) transforma exponencial em linear.

5. Dividir para conquistar

5.1 A estratégia

Em uma frase: quebrar um problema grande em problemas menores **do mesmo tipo**, resolver cada um (recursivamente) e juntar as respostas.

As três etapas:

1. **Dividir:** partir o problema em subproblemas menores.
2. **Conquistar:** resolver cada subproblema recursivamente, até chegar ao **caso base** (o caso mais simples possível).
3. **Combinar:** juntar as soluções dos subproblemas na solução do problema original.

A receita para criar um algoritmo assim: (1) descubra o caso base; (2) descubra como reduzir o problema até ele virar o caso base.

Algoritmos vistos que usam dividir para conquistar:

Algoritmo	Como divide	Caso base	Como combina
Busca binária	descarta metade da lista	intervalo vazio ou item no meio	não precisa: a resposta vem da metade que sobrou
Quicksort	separa menores e maiores que o pivô	lista com 0 ou 1 item	menores + pivô + maiores
Merge sort	corta ao meio	lista com 0 ou 1 item	intercala as duas metades
Euclides (fazenda)	troca o terreno pela sobra	um lado é múltiplo do outro	não precisa: a resposta da sobra é a resposta do todo
Soma recursiva de lista	tira o primeiro item	lista vazia	primeiro item + soma do resto

5.2 Euclides: o maior quadrado da fazenda (MDC)

Problema do livro: um terreno de 1680×640 m deve ser dividido em lotes **quadrados iguais**, os maiores possíveis. Qual o tamanho do lote?

Em uma frase: encaixe os maiores quadrados possíveis no terreno; o que sobrar é um terreno menor, e o maior quadrado que cabe na **sobra** é o maior que serve para o **terreno todo**. Repita até que um lado seja múltiplo do outro.

Como funciona:

1. **Caso base:** se um lado é múltiplo do outro (resto da divisão = 0), o lote tem o tamanho do lado menor.
2. **Caso recursivo:** senão, calcule a sobra (lado maior **mod** lado menor) e resolva o problema para o terreno (lado menor $\times$ sobra).

Exemplo:

```
(1680, 640): 1680 mod 640 = 400 -> sobra 640 x 400
( 640, 400):  640 mod 400 = 240 -> sobra 400 x 240
( 400, 240):  400 mod 240 = 160 -> sobra 240 x 160
( 240, 160):  240 mod 160 =  80 -> sobra 160 x  80
( 160,  80):  160 mod  80 =   0 -> CASO BASE: lote de 80 x 80
```

Esse é o **algoritmo de Euclides** para o máximo divisor comum: $\text{MDC}(1680, 640) = 80$. Em Python: `math.gcd(1680, 640)`.

Custo: a cada dois passos o lado menor pelo menos cai pela metade, então são cerca de log do menor lado passos: **$O(\log n)$** . Rapidíssimo mesmo para números enormes.

6. Força bruta: o caixeiro-viajante

Problema: um vendedor precisa visitar n cidades, cada uma exatamente uma vez, e voltar ao início, pelo caminho mais curto.

Em uma frase (força bruta): liste **todas** as ordens possíveis de visitar as cidades, calcule o comprimento de cada uma e escolha a menor.

Custo: há $n!$ ordens possíveis (fatorial). Para 5 cidades, 120 rotas; para 10, 3,6 milhões; para 15, 1,3 trilhão; para 20, mais de 2 quintilhões. Cada cidade a mais **multiplica** o trabalho por n . Classe **$O(n!)$** , a mais lenta da tabela: com 16 cidades e um computador que faz 10 operações por segundo, o livro estima 66 mil anos.

Por que aparece no curso: é o exemplo de problema para o qual **não se conhece algoritmo rápido**. Na prática usam-se aproximações (heurísticas) que dão uma boa rota, mas não garantem a melhor.

7. Comparativo geral

7.1 Quanto tempo cada classe leva, na prática

Supondo um computador que faz **10 milhões de operações simples por segundo**:

Classe	Exemplo	n = 1.000	n = 1.000.000	Se n dobra...
O(1)	acessar lista[i]	instantâneo	instantâneo	igual
O(log n)	busca binária	10 op → instantâneo	20 op → instantâneo	+1 operação
O(n)	busca linear, Fibonacci iterativo	1.000 op → 0,0001 s	1 milhão op → 0,1 s	dobra
O(n log n)	quicksort, merge sort	10.000 op → 0,001 s	20 milhões op → 2 s	pouco mais que dobra
O(n²)	bubble, selection, quicksort no pior caso	1 milhão op → 0,1 s	1 trilhão op → ≈ 28 horas	quadruplica
O(2 ⁿ)	Fibonacci recursivo puro	2 ¹⁰⁰⁰ op → nunca termina	—	eleva ao quadrado
O(n!)	caixeiro-viajante por força bruta	1000! op → nunca termina	—	multiplica por n

Leitura: até O(n log n), tudo é viável mesmo para milhões de itens. Em O(n²), mil itens ainda são tranquilos, mas um milhão já leva um dia. Exponencial e fatorial só funcionam para n muito pequeno (até uns 25 a 30 para 2ⁿ, até uns 10 a 12 para n!).

7.2 Qual algoritmo usar em cada situação

Se você precisa...	Use	Por quê
achar um item em lista pequena ou desordenada, uma vez	busca linear	não exige preparo; ordenar custaria mais
achar itens muitas vezes em uma lista grande	ordenar uma vez e usar busca binária	O(log n) por busca
ordenar poucos itens ou uma lista quase ordenada	bubble sort com parada	para cedo, tende a O(n)
ordenar poucos itens quando trocar é caro	selection sort	no máximo n-1 trocas
ordenar muitos itens em geral	quicksort com pivô aleatório ou do meio	O(n log n) e rápido na prática
ordenar com pior caso garantido ou mantendo a ordem de empates	merge sort	O(n log n) sempre; estável
calcular algo com subproblemas repetidos (Fibonacci e parecidos)	memorização ou versão iterativa	O(n) em vez de O(2 ⁿ)
calcular F(n) para muitos n ao longo do programa	memorização com cache persistente	consultas repetidas custam O(1)
maior divisor comum / maior quadrado	algoritmo de Euclides	O(log n)
testar todas as combinações possíveis	força bruta, só se n for pequeno	O(n!) ou O(2 ⁿ) explodem

7.3 Os mesmos algoritmos, agrupados por ideia

Ideia central	Algoritmos	O que têm em comum
Olhar tudo, um por um	busca linear, selection sort, bubble sort, Fibonacci iterativo	simples; custo proporcional a n (ou n^2 se for “um por um” dentro de “um por um”)
Cortar o problema pela metade	busca binária, merge sort, quicksort (com bom pivô), Euclides	cada passo reduz muito o problema; aparece o $\log n$
Não refazer o que já foi feito	Fibonacci com memorização, Fibonacci iterativo, bubble sort com parada	reaproveitar resultados ou perceber que já acabou
Tentar todas as possibilidades	caixeiro-viajante por força bruta, Fibonacci recursivo puro	corretos, porém exponenciais ou fatoriais: inviáveis para n grande

7.4 Cinco frases para levar

1. **Big O mede crescimento, não segundos.** Dois algoritmos $O(n)$ podem ter velocidades diferentes; o que o Big O diz é como cada um reage quando n cresce.
2. **Dividir pela metade gera $\log n$.** Sempre que um algoritmo corta o problema ao meio a cada passo (busca binária, merge sort, quicksort com bom pivô), aparece o $\log n$, e o algoritmo escala muito bem.
3. **Laço dentro de laço gera n^2 .** Selection e bubble comparam cada item com cada outro; por isso quadruplicam quando n dobra.
4. **Recalcular o mesmo subproblema gera exponencial.** Fibonacci recursivo puro é o exemplo; um cache resolve.
5. **O melhor algoritmo depende da entrada e do uso.** Lista quase ordenada favorece o bubble; lista ordenada com pivô fixo arruína o quicksort; uma busca só favorece a linear, mil buscas favorecem a binária.

Fontes: aulas 03 a 06 (Prof. Humberto Zanetti, Fatec Jundiaí); atividades 01, 02 e 03; BHARGAVA, Aditya Y. Entendendo Algoritmos, caps. 1 a 4. Os números das tabelas vêm das medições feitas nas atividades e nos scripts do Catálogo.